



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 12

Control de LED RGB mediante Señales PWM

Carrera:

Ingeniería Mecatrónica — Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

19 de Marzo de 2026

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Trabajo Previo	2
1.2.1. Señal PWM — Definición y Parámetros	2
1.2.2. PWM en el RP2040	2
2. Desarrollo	3
2.1. Diseño del Circuito	3
2.2. Descripción del Sistema	4
2.3. Descripción de Dificultades y Soluciones	5
3. Preguntas de Repaso	5
3.1. ¿Qué es el ciclo de trabajo en una señal PWM?	5
3.2. ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?	6
3.3. ¿Cómo se genera una señal PWM en un microcontrolador?	6
3.4. ¿Qué efecto tiene variar la frecuencia de la señal PWM en la iluminación del LED RGB?	6
3.5. ¿Cuál es la relación entre la resolución del ADC y la precisión del control PWM?	6
3.6. ¿Cómo se pueden mejorar las transiciones de color en el LED RGB?	6
3.7. ¿Qué aplicaciones prácticas tiene el uso de PWM en sistemas de control?	7
3.8. ¿Cuál es la diferencia entre PWM y modulación por amplitud de pulso (PAM)?	7
4. Conclusión	7
Anexos	9

1. Introducción

En esta práctica se estudia el principio de operación de las señales de **Modulación por Ancho de Pulso** (PWM, *Pulse Width Modulation*) y su aplicación en el control de intensidad lumínica de un **LED RGB** mediante la Raspberry Pi Pico W. El PWM es una de las técnicas más empleadas en sistemas embebidos para simular una salida analógica con recursos puramente digitales: variando la proporción de tiempo en que la señal permanece en estado alto (ciclo de trabajo) dentro de un período fijo, se controla la energía promedio entregada a la carga.

En el caso de los LEDs, la alta frecuencia de conmutación supera la velocidad de respuesta del ojo humano, por lo que la variación de brillo percibida resulta continua y sin parpadeo. Al disponer de tres canales PWM independientes —uno para cada color primario del LED RGB (rojo, verde y azul)— es posible mezclar colores de forma aditiva y obtener cualquier tono dentro del espacio de color de 24 bits.

El sistema implementado incluye tres potenciómetros conectados a los canales ADC de la Pico W para determinar analógicamente el nivel de cada color, tres canales PWM de 8 bits para controlar el LED RGB, transmisión serial del valor porcentual de cada canal vía USB-CDC, y una interfaz gráfica en Python que muestra en tiempo real los valores leídos, la mezcla de color resultante y una gráfica histórica de los tres canales.

1.1. Objetivo

Aprender el funcionamiento de las señales PWM del microcontrolador como método de control en dispositivos como los LEDs RGB, y comprender la relación entre el ciclo de trabajo de la señal PWM y la intensidad luminosa percibida.

1.2. Trabajo Previo

1.2.1. Señal PWM — Definición y Parámetros

Una señal PWM es una onda cuadrada de **período constante** T cuyo tiempo en nivel alto, denominado **ancho de pulso** t_{on} , varía entre 0 y T . El parámetro más relevante para el control es el **ciclo de trabajo** D :

$$D [\%] = \frac{t_{on}}{T} \times 100 \quad (1)$$

La tensión promedio entregada a la carga es directamente proporcional al ciclo de trabajo:

$$V_{prom} = D \times V_{alto} \quad (2)$$

1.2.2. PWM en el RP2040

El microcontrolador RP2040 integra **8 bloques PWM** (*slices*), cada uno con dos canales independientes (A y B). La frecuencia de la señal generada se calcula como:

$$f_{PWM} = \frac{f_{clk}}{(WRAP + 1) \times DIV} \quad (3)$$

donde $f_{clk} = 125 \text{ MHz}$, $WRAP$ es el valor máximo del contador (255 para 8 bits) y DIV es el divisor de reloj configurado con `pwm_set_clkdiv()`.

2. Desarrollo

2.1. Diseño del Circuito

El circuito integra el bloque de **entradas analógicas** (tres potenciómetros) y el bloque de **salidas PWM** (LED RGB).

- **Potenciómetros** ($5 \text{ k}\Omega$ + resistencia de protección $1 \text{ k}\Omega$): un terminal a 3.3 V , el otro a GND y el cursor al pin ADC.

Canal	Pin ADC	Canal ADC
Rojo (R)	GP28	ADC 2
Verde (G)	GP26	ADC 0
Azul (B)	GP27	ADC 1

- **LED RGB (cátodo común)**: cada ánodo conectado mediante una resistencia de 220Ω al pin PWM correspondiente; el cátodo común a GND.

Canal	Pin PWM	Resistencia
Rojo (R)	GP15	220Ω
Verde (G)	GP14	220Ω
Azul (B)	GP13	220Ω

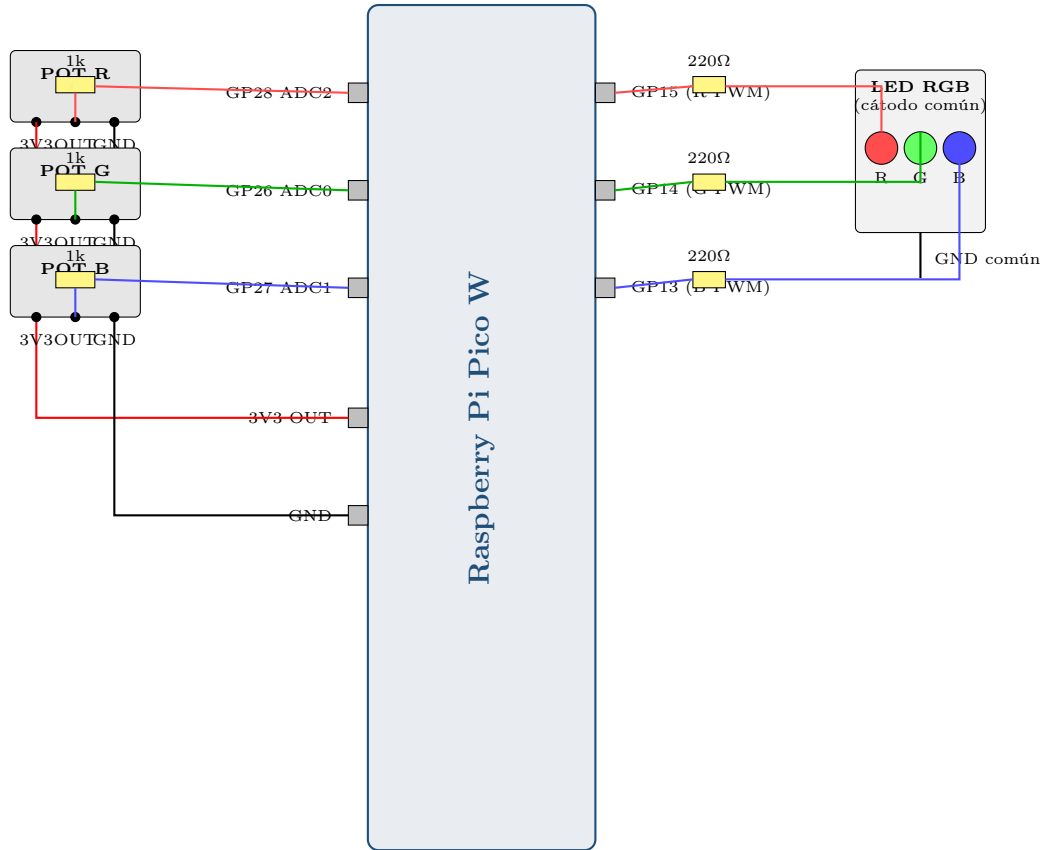


Figura 1: Diagrama esquemático: potenciómetros ($5\text{ k}\Omega + 1\text{ k}\Omega$) en GP28/GP26/GP27 como entradas ADC; LED RGB de cátodo común con resistencias de 220Ω en GP15/GP14/GP13 como salidas PWM.

2.2. Descripción del Sistema

El sistema opera de manera continua según el siguiente ciclo:

1. **Lectura ADC:** Se miden los tres canales secuencialmente con un promedio de 8 muestras por canal (separadas $10\mu\text{s}$) para reducir el ruido de cuantización.
2. **Cálculo del ciclo de trabajo:** El valor ADC de 12 bits ($0-4095$) se mapea al rango de 8 bits del PWM ($0-255$):

$$D_{PWM} = \left\lfloor \frac{ADC \times 255}{4095} \right\rfloor \quad (4)$$

3. **Actualización PWM:** Se escribe el nivel en cada canal con `pwm_set_chan_level()`. Para LED de ánodo común se invierte: $D = 255 - D_{PWM}$.
4. **Transmisión serial (cada 80 ms):**

DATA:r,g,b,MODE y R: XX% | G: XX% | B: XX%

Se soportan dos modos: **MANUAL** (potenciómetros) y **SERIAL** (comando `RGB:r,g,b` desde la PC).

2.3. Descripción de Dificultades y Soluciones

Cuadro 1: Errores encontrados y soluciones aplicadas

Dificultad	Solución
El LED enciende con color incorrecto o lógica invertida.	Se verificó el tipo de LED (cátodo o ánodo común). Para ánodo común la lógica se invierte con la macro <code>LED_CATODO_COMUN = 0</code> , calculando el nivel como $255 - D$.
Los valores en el monitor serial oscilan aunque el potenciómetro esté fijo.	Se implementó un promedio de 8 muestras ADC por canal con <code>sleep_us(10)</code> entre lecturas para reducir el ruido de cuantización.
La interfaz Python se congela al leer el puerto serial.	La lectura serial se movió a un hilo <i>daemon</i> con <code>threading.Thread(daemon=True)</code> . Las actualizaciones de la GUI se realizan con <code>root.after(0, callback)</code> para evitar condiciones de carrera.
Parpadeo visible en el LED.	Con <code>WRAP = 255</code> y el divisor de reloj predeterminado se obtiene $f_{PWM} \approx 1 \text{ kHz}$, frecuencia suficientemente alta para que el parpadeo sea imperceptible.

3. Preguntas de Repaso

3.1. ¿Qué es el ciclo de trabajo en una señal PWM?

El **ciclo de trabajo** (*duty cycle*) es la fracción de tiempo dentro de un período de la señal PWM durante la cual la señal está en nivel alto:

$$D [\%] = \frac{t_{on}}{T} \times 100 \quad (5)$$

Un ciclo de trabajo del 0 % significa tensión promedio nula; al 100 % equivale a una señal DC de V_{alto} . Los valores intermedios producen una tensión promedio $V_{prom} = D \times V_{alto}$, permitiendo controlar potencia de forma continua con señales digitales.

3.2. ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?

Significa que la señal está en nivel alto el 40 % del período y en bajo el 60 % restante. Para $V_{alto} = 3,3 \text{ V}$, la tensión promedio es: $V_{prom} = 0,40 \times 3,3 = 1,32 \text{ V}$. En el LED, el ojo percibe un brillo equivalente al 40 % del máximo siempre que la frecuencia supere los 50 Hz.

3.3. ¿Cómo se genera una señal PWM en un microcontrolador?

En el RP2040 los pasos son: (1) asociar el pin con `gpio_set_function(pin, GPIO_FUNC_PWM)`; (2) obtener el *slice* y canal con `pwm_gpio_to_slice_num()` y `pwm_gpio_to_channel()`; (3) establecer el período con `pwm_set_wrap(slice, WRAP)`; (4) ajustar el ciclo de trabajo con `pwm_set_chan_level(slice, ch, level)`; y (5) habilitar el *slice* con `pwm_set_enabled(slice, true)`. Una vez activo, el periférico opera de forma autónoma sin consumir ciclos de CPU.

3.4. ¿Qué efecto tiene variar la frecuencia de la señal PWM en la iluminación del LED RGB?

A frecuencias bajas ($< 50 \text{ Hz}$) el parpadeo es perceptible. Entre 50 Hz y 1 kHz el brillo percibido es estable y proporcional al ciclo de trabajo (rango estándar para iluminación). A frecuencias muy altas pueden aumentar las pérdidas de conmutación e interferencias electromagnéticas sobre circuitos analógicos del sistema. El color percibido no cambia con la frecuencia mientras el ciclo de trabajo permanezca constante.

3.5. ¿Cuál es la relación entre la resolución del ADC y la precisión del control PWM?

La resolución efectiva del sistema la limita el componente de menor número de bits. Con ADC de 12 bits y PWM de 8 bits, el factor limitante es el PWM: $\Delta D = 100 \text{ \%}/255 \approx 0,39 \text{ \%}$ por nivel. Aumentar el WRAP a 4095 (12 bits) mejoraría la resolución a $\approx 0,024 \text{ \%}$ por nivel, aprovechando íntegramente la precisión del ADC y produciendo transiciones de color más suaves.

3.6. ¿Cómo se pueden mejorar las transiciones de color en el LED RGB?

Las estrategias principales son: aumentar la resolución PWM a 12 bits (4096 niveles); aplicar **corrección gamma** ($\gamma = 2,2$) para que los incrementos de brillo sean perceptualmente uniformes; implementar una **rampa de interpolación** entre el valor actual y el objetivo en varios pasos intermedios; filtrar las lecturas ADC con una media móvil para suavizar variaciones bruscas del potenciómetro; y controlar el LED en espacio **HSV** para generar barridos de color naturales.

3.7. ¿Qué aplicaciones prácticas tiene el uso de PWM en sistemas de control?

Sus principales aplicaciones son: control de velocidad y par en motores DC y servomotores; reguladores de tensión conmutados (*buck/boost*); control de brillo en iluminación LED y retroiluminación de pantallas LCD; amplificadores de audio clase D con eficiencias superiores al 90 %; control proporcional de resistencias calefactoras en sistemas HVAC; cargadores de baterías de litio; y control de múltiples actuadores en robótica móvil.

3.8. ¿Cuál es la diferencia entre PWM y modulación por amplitud de pulso (PAM)?

Cuadro 2: Comparación entre PWM y PAM

Característica	PWM	PAM
Parámetro modulado	Ancho del pulso	Amplitud del pulso
Amplitud	Constante (V_{alto} fijo)	Variable
Inmunidad al ruido	Alta: el ruido afecta amplitud, no ancho	Baja: el ruido se suma a la amplitud
Implementación	Sencilla con hardware digital	Requiere DAC o circuitos analógicos
Eficiencia	Alta: transistor en saturación o corte	Media
Aplicación típica	Control de potencia, motores, LEDs	Telefonía analógica, Ethernet

4. Conclusión

Esta práctica consolidó la comprensión práctica de la modulación por ancho de pulso como mecanismo de control analógico a partir de recursos puramente digitales. Los periféricos PWM del RP2040 operan de forma autónoma una vez configurados, sin consumir ciclos de CPU, lo que permite atender simultáneamente la lectura del ADC, la comunicación serial y la lógica de control.

La resolución de 8 bits del PWM resultó adecuada para producir transiciones de color visualmente continuas. Se verificó experimentalmente que la mezcla aditiva de los tres canales produce los colores esperados según el modelo RGB, y que el sistema responde correctamente tanto en modo MANUAL como en modo SERIAL.

Posibles mejoras: aumentar el PWM a 12 bits para aprovechar la resolución completa del ADC; implementar corrección gamma para transiciones de brillo perceptual-

mente lineales; control en espacio HSV con los potenciómetros asociados a tono, saturación y brillo; y aprovechar el módulo Wi-Fi de la Pico W para recibir comandos RGB desde una aplicación web.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.2). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [3] pySerial Contributors, *pySerial Documentation*. <https://pyserial.readthedocs.io>, 2024.
- [4] Matplotlib Development Team, *Matplotlib: Visualization with Python*. <https://matplotlib.org/stable/>, 2024.

Anexos

Anexo A — Código Fuente: Practica12.c

```

/*
 * Practica 12 — Control LED RGB mediante senales PWM
 * Framework : pico-sdk | Placa: Raspberry Pi Pico W
 *
 * Entradas ADC (potenciometros 5k + 1k):
 *   Red    -> GP28 (ADC2)
 *   Green  -> GP26 (ADC0)
 *   Blue   -> GP27 (ADC1)
 *
 * Salidas PWM (LED RGB catodo comun, res. 220 Ohm):
 *   Red    -> GP15
 *   Green  -> GP14
 *   Blue   -> GP13
 *
 * Protocolo salida:
 *   "DATA:r,g,b,MODE\n"           reporte periodico
 *   "Canales de color:\n"
 *   "R: XX% / G: XX% / B: XX%\n"
 *
 * Protocolo entrada:
 *   "MODE:MANUAL\n" / "MODE:SERIAL\n"
 *   "RGB:r,g,b\n"   (solo en modo SERIAL)
 *   "GET\n"
 */

#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pwm.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define POT_R_PIN    28
#define POT_G_PIN    26
#define POT_B_PIN    27
#define LED_R_PIN    15
#define LED_G_PIN    14
#define LED_B_PIN    13

#define ADC_MAX      4095u
#define PWM_WRAP     255u

```

```
#define SAMPLE_MS      80u
#define CMD_BUF_SIZE   64
#define ADC_PROMEDIO   8

/* 1 = catodo comun | 0 = anodo comun */
#define LED_CATODO_COMUN 1

typedef enum { MODE_MANUAL = 0, MODE_SERIAL = 1 } CtrlMode;

static CtrlMode g_mode = MODE_MANUAL;
static uint8_t g_red = 0;
static uint8_t g_green = 0;
static uint8_t g_blue = 0;

/* ————— Helpers PWM ————— */
static void pwm_pin_init(uint pin) {
    gpio_set_function(pin, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(pin);
    pwm_set_wrap(slice, PWM_WRAP);
    pwm_set_enabled(slice, true);
}

static void pwm_set_level(uint pin, uint8_t level) {
    uint slice = pwm_gpio_to_slice_num(pin);
    uint channel = pwm_gpio_to_channel(pin);
    #if LED_CATODO_COMUN
        pwm_set_chan_level(slice, channel, level);
    #else
        pwm_set_chan_level(slice, channel, PWM_WRAP - level);
    #endif
}

static void apply_color(void) {
    pwm_set_level(LED_R_PIN, g_red);
    pwm_set_level(LED_G_PIN, g_green);
    pwm_set_level(LED_B_PIN, g_blue);
}

/* ————— Lectura ADC con promedio ————— */
static uint8_t read_adc_channel(uint channel) {
    adc_select_input(channel);
    sleep_us(10);
    uint32_t suma = 0;
    for (int i = 0; i < ADC_PROMEDIO; i++) {
        suma += adc_read();
    }
}
```

```

        sleep_us(10);
    }
    uint16_t prom = (uint16_t)(suma / ADC_PROMEDIO);
    return (uint8_t)((prom * 255u) / ADC_MAX);
}

/* ————— Parser de comandos serial ————— */
static void parse_command(const char *cmd) {

    if (strncmp(cmd, "MODE:", 5) == 0) {
        const char *arg = cmd + 5;
        if (strcmp(arg, "MANUAL") == 0) {
            g_mode = MODE_MANUAL;
            printf("ACK:MODE:MANUAL\n");
        } else if (strcmp(arg, "SERIAL") == 0) {
            g_mode = MODE_SERIAL;
            printf("ACK:MODE:SERIAL\n");
        } else {
            printf("ERR:UNKNOWN_MODE%s\n", arg);
        }
        return;
    }

    if (strncmp(cmd, "RGB:", 4) == 0) {
        int r, g, b;
        if (sscanf(cmd + 4, "%d,%d,%d", &r, &g, &b) == 3) {
            if (r >= 0 && r <= 255 && g >= 0 && g <= 255 && b >= 0 && b <= 255) {
                if (g_mode == MODE_SERIAL) {
                    g_red = (uint8_t)r;
                    g_green = (uint8_t)g;
                    g_blue = (uint8_t)b;
                    apply_color();
                    printf("ACK:RGB:%d,%d,%d\n", r, g, b);
                } else {
                    printf("ERR:NOT_IN_SERIAL_MODE\n");
                }
            } else {
                printf("ERR:VALUE_OUT_OF_RANGE\n");
            }
        } else {
            printf("ERR:INVALID_RGB_FORMAT\n");
        }
        return;
    }
}

```

```

    if (strcmp(cmd, "GET") == 0) {
        printf("ACK:GET:%d,%d,%d,%s\n",
            g_red, g_green, g_blue,
            g_mode == MODE_MANUAL ? "MANUAL" : "SERIAL");
        return;
    }

    printf("ERR:UNKNOWN_CMD %s\n", cmd);
}

/* ----- main ----- */
int main(void) {

    stdio_init_all();
    sleep_ms(2000);    /* Esperar enumeracion USB */

    adc_init();
    adc_gpio_init(POT_R_PIN);
    adc_gpio_init(POT_G_PIN);
    adc_gpio_init(POT_B_PIN);

    pwm_pin_init(LED_R_PIN);
    pwm_pin_init(LED_G_PIN);
    pwm_pin_init(LED_B_PIN);
    apply_color();

    char    cmd_buf[CMD_BUF_SIZE];
    uint8_t cmd_idx = 0;
    uint32_t last_ms = 0;

    printf("INIT:RGB_CONTROLLER:READY\n");
    printf("INFO:PINS:R=GP15,G=GP14,B=GP13\n");
    printf("INFO:POTS:R=GP28,G=GP26,B=GP27\n");
    printf("INFO:MODE:MANUAL\n");

    while (true) {

        uint32_t now = to_ms_since_boot(get_absolute_time());

        /* Lectura serial no bloqueante */
        int c = getchar_timeout_us(0);
        if (c != PICO_ERROR_TIMEOUT) {
            char ch = (char)c;
            if (ch == '\n' || ch == '\r') {
                if (cmd_idx > 0) {

```

```

        cmd_buf[cmd_idx] = '\0';
        parse_command(cmd_buf);
        cmd_idx = 0;
    }
} else if (cmd_idx < CMD_BUF_SIZE - 1) {
    cmd_buf[cmd_idx++] = ch;
}
}

/* Muestreo periodico */
if (now - last_ms >= SAMPLE_MS) {
    last_ms = now;

    if (g_mode == MODE_MANUAL) {
        g_red    = read_adc_channel(2);    /* GP28 = ADC2 */
        g_green  = read_adc_channel(0);    /* GP26 = ADC0 */
        g_blue   = read_adc_channel(1);    /* GP27 = ADC1 */
        apply_color();
    }

    uint8_t pct_r = (uint8_t)((g_red    * 100u) / 255u);
    uint8_t pct_g = (uint8_t)((g_green  * 100u) / 255u);
    uint8_t pct_b = (uint8_t)((g_blue   * 100u) / 255u);

    printf("DATA:%d,%d,%d,%s\n",
           g_red, g_green, g_blue,
           g_mode == MODE_MANUAL ? "MANUAL" : "SERIAL");

    printf("Canales_de_color:\n");
    printf("R:_%3d%% | _G:_%3d%% | _B:_%3d%%\n",
           pct_r, pct_g, pct_b);
}

sleep_ms(1);
}

return 0;
}

```

Anexo B — Evidencia Fotográfica

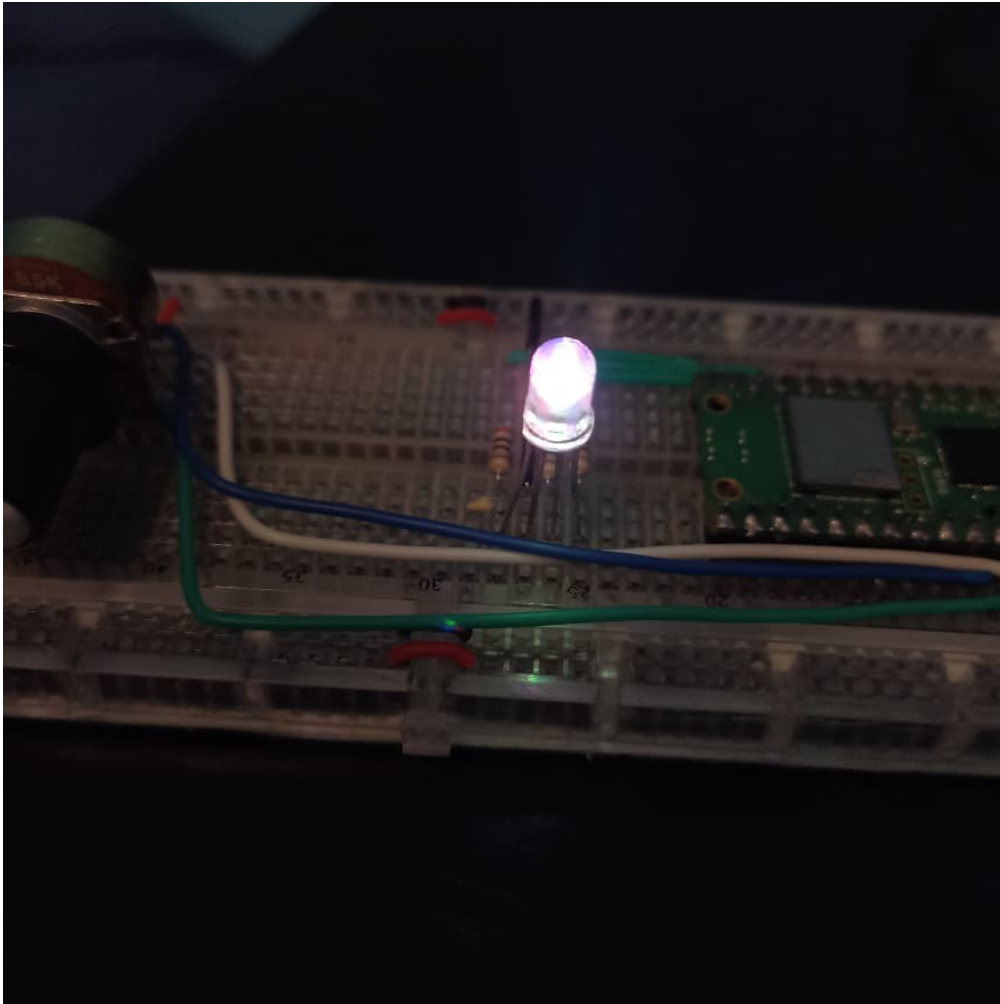


Figura 2: Prototipo físico del circuito de control de LED RGB mediante PWM armado en protoboard.



Figura 3: Interfaz gráfica Python ejecutándose en la computadora, mostrando en tiempo real los valores RGB del Raspberry Pi Pico W vía USB-serial.